So, back to the constructor of `data_to_reclaim` ❶: the constructor is also a template. It stores the data to be deleted as a `void*` in the `data` member and then stores a pointer to the appropriate instantiation of `do_delete()`—a simple function that casts the supplied `void*` to the chosen pointer type and then deletes the pointed-to object. `std::function<>` wraps this function pointer safely, so that the destructor of `data_to_reclaim` can then delete the data just by invoking the stored function ❷.

The destructor of `data_to_reclaim` isn't called when you're adding nodes to the list; it's called when there are no more hazard pointers to that node. This is the responsibility of `delete_nodes_with_no_hazards()`.

`delete_nodes_with_no_hazards()` first claims the entire list of nodes to be reclaimed for itself with a simple `exchange()` ❻. This simple but crucial step ensures that this is the only thread trying to reclaim this particular set of nodes. Other threads are now free to add further nodes to the list or even try to reclaim them without impacting the operation of this thread.

Then, as long as there are still nodes left in the list, you check each node in turn to see if there are any outstanding hazard pointers ❼. If there aren't, you can safely delete the entry (and thus clean up the stored data) ❽. Otherwise, you just add the item back on the list for reclaiming later ❾.

Although this simple implementation does indeed safely reclaim the deleted nodes, it adds quite a bit of overhead to the process. Scanning the hazard pointer array requires checking `max_hazard_pointers` atomic variables, and this is done for every `pop()` call. Atomic operations are inherently slow—often 100 times slower than an equivalent non-atomic operation on desktop CPUs—so this makes `pop()` an expensive operation. Not only do you scan the hazard pointer list for the node you're about to remove, but you also scan it for each node in the waiting list. Clearly this is a bad idea. There may well be `max_hazard_pointers` nodes in the list,